

Container Runtime, OCI, containerd & runc

Qui met réellement en place les mécanismes Linux quand on lance un conteneur ?

Version de révision : schémas explicatifs · termes expliqués · points clés · corrigé des questions

Objectif

Comprendre le rôle d'un **container runtime**.

Idée clé

Le chapitre précédent expliquait **comment Linux isole un processus** ; ce chapitre explique **quels composants utilisent ces mécanismes** pour exécuter et gérer des conteneurs.

La chaîne à connaître

Docker CLI → dockerd → containerd → containerd-shim → runc → **Linux Kernel**

Et chez Podman : podman → conmon → runc / crun → **Linux Kernel**

Comment lire ce document

Bloc	Signification
En clair	Explication simple d'un terme important, avec une analogie.
À retenir	Les points clés de la section, à connaître par cœur.
Attention	Pièges, nuances et précisions importantes.
Blocs de code	Commandes ou schémas à reproduire dans un terminal Linux.

Sommaire

1. Du Linux Kernel au Container Runtime	3
2. Qu'est-ce qu'un Container Runtime ?	3
3. Types de Runtime	4
4. Pourquoi cette séparation ?	5
5. Architecture Docker	6
6. Architecture Podman	10
7. Différence entre Docker et Podman	12
8. OCI — Open Container Initiative	13
Ce qu'il faut absolument retenir	15
Glossaire des termes importants	16
Questions de compréhension — avec corrigés	17
Préparation pour la suite	18

1. Du Linux Kernel au Container Runtime

Dans le chapitre précédent, nous avons étudié les mécanismes fondamentaux utilisés par les conteneurs :

```
Linux Kernel
├── Namespaces
├── cgroups
├── mounts
├── rootfs
└── Process
```

Nous savons donc maintenant comment Linux peut isoler et contrôler un processus. Mais une question reste :

La question

Quel logiciel met en place tous ces mécanismes lorsqu'on demande de lancer un conteneur ?

C'est le rôle du **container runtime**.

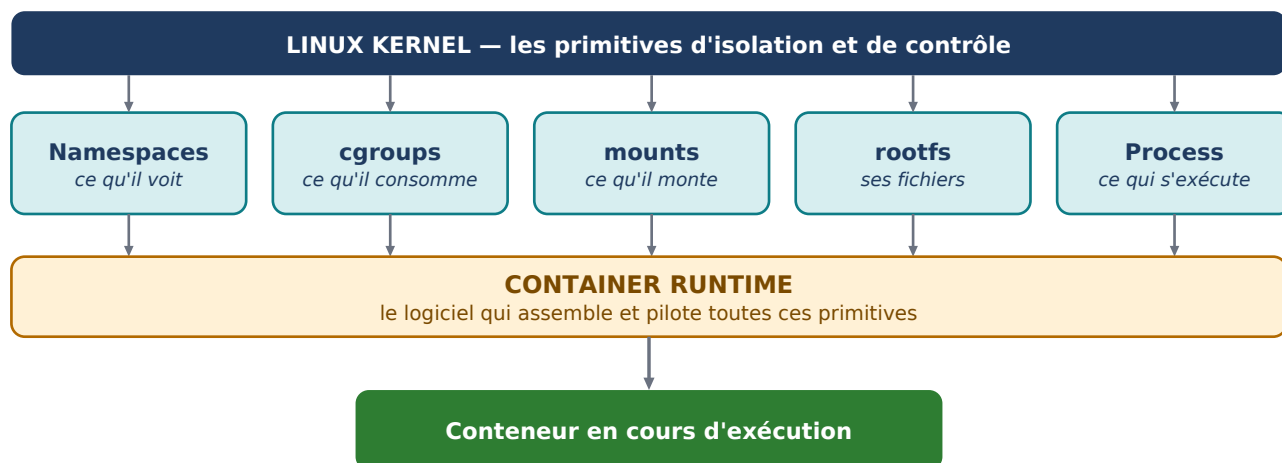


Figure 1 — Le kernel fournit les briques ; le container runtime les assemble pour fabriquer un conteneur.

En clair — Les primitives du kernel

Le kernel fournit les **briques et les outils d'un chantier** (isoler, limiter, monter, exécuter). Mais quelqu'un doit les assembler dans le bon ordre pour construire la maison. Ce quelqu'un, c'est le container runtime.

À retenir

- ✓ Le kernel fournit les mécanismes (namespaces, cgroups, mounts...) mais ne « lance » pas de conteneur tout seul.
- ✓ Le **container runtime** est le logiciel qui utilise ces mécanismes pour créer et gérer les conteneurs.

2. Qu'est-ce qu'un Container Runtime ?

Un **container runtime** est un logiciel responsable de la **gestion et/ou de l'exécution** des conteneurs, en utilisant les mécanismes fournis par le système d'exploitation.

Le terme est assez général. Dans l'écosystème moderne, on rencontre notamment **deux niveaux** :

```
High-Level: Gestion du conteneur
Low-Level: Exécution du processus
```

En clair — Container runtime

Pensez à un **chantier**. Le **chef de chantier** (high-level) organise : commande les matériaux, suit le planning, coordonne les équipes. Le **maçon** (low-level) construit réellement le mur, brique par brique. Les deux sont indispensables, mais ils n'ont pas le même métier.

Un mot, plusieurs sens

Dans le langage courant, « runtime » désigne parfois seulement **runc**, parfois tout Docker. Pour éviter la confusion, on précise toujours le **niveau** : high-level (gérer) ou low-level (exécuter).

3. Types de Runtime

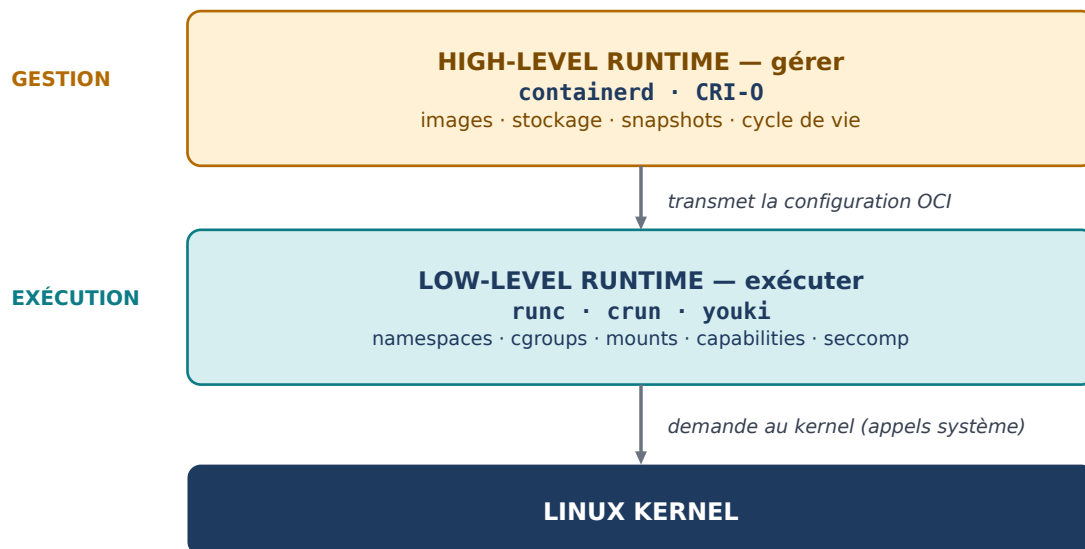


Figure 2 — Deux niveaux de runtime au-dessus du kernel.

3.1. High-Level Runtime

Un **high-level container runtime** gère les aspects généraux du conteneur, par exemple :

- les **images** ;
- le **stockage** du contenu ;
- les **snapshots** ;
- le **cycle de vie** ; ...

Exemples : **containerd**, **CRI-O**.

High-level runtime

= **gérer et orchestrer** le cycle de vie du conteneur.

3.2. Low-Level Runtime

Un **low-level container runtime** est beaucoup plus proche du système d'exploitation et s'occupe de **créer et exécuter le processus** du conteneur.

Exemples : **runc**, **crun**, **youki**.

Il utilise notamment : **namespaces**, **cgroups**, **mounts**, **capabilities** et **seccomp**.

Low-level runtime

= **créer, exécuter et isoler** le processus du conteneur.

3.3. Comparaison

	High-level runtime	Low-level runtime
Rôle	Gérer et orchestrer le cycle de vie	Créer, exécuter et isoler le processus
Gère	images, stockage, snapshots, tâches, cycle de vie	namespaces, cgroups, mounts, capabilities, seccomp
Proche de	l'utilisateur et de l'orchestrateur	du système (kernel)
Exemples	containerd, CRI-O	runc, crun, youki

3.4. Mots à connaître

Terme	En clair
Snapshot	Photo instantanée de l'état d'un système de fichiers. Les runtimes s'en servent pour préparer le filesystem d'un conteneur à partir des couches d'une image.
Layer (couche)	Une image est un empilement de couches en lecture seule ; chaque couche ajoute ou modifie quelques fichiers.
Cycle de vie	Les étapes de l'existence d'un conteneur : créé → démarré → en cours → arrêté → supprimé.
Capabilities	Le tout-puissant « root » découpé en petits droits précis (ex. : écouter sur un port bas, changer l'heure). On ne donne au conteneur que ceux dont il a besoin.
seccomp	Un filtre qui décide quels appels système un processus a le droit de faire. Tout appel non autorisé est bloqué.

À retenir

- ✓ **High-level** = images, stockage, snapshots, cycle de vie → [containerd](#), [CRI-O](#).
- ✓ **Low-level** = création et exécution du processus avec le kernel → [runc](#), [crun](#), [youki](#).

4. Pourquoi cette séparation ?

Prenons la commande :

```
docker run nginx
```

Derrière cette commande, plusieurs opérations doivent être réalisées :

1. Trouver l'image
2. Télécharger l'image si nécessaire
3. Stocker ses layers
4. Préparer le filesystem
5. Créer le conteneur
6. Préparer le réseau
7. Configurer l'isolation
8. Configurer les ressources
9. Lancer le processus

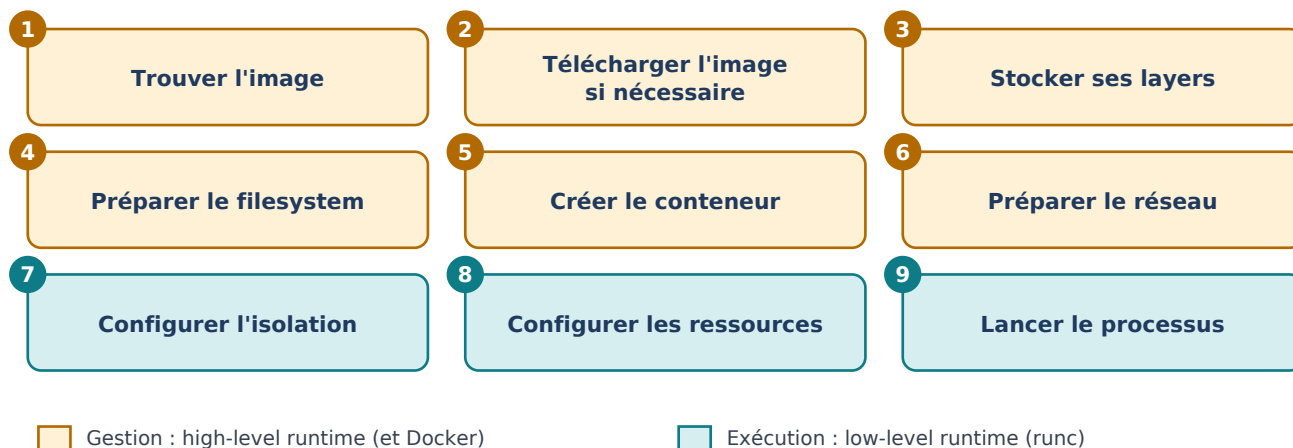


Figure 3 — Les neuf opérations cachées derrière « `docker run nginx` » (répartition simplifiée).

Il est donc logique de **séparer les responsabilités** :

```
High-Level Runtime
|
▼
Low-Level Runtime
|
▼
Linux Kernel
```

Cette séparation nous permet ensuite de comprendre pourquoi `containerd` et `runc`, par exemple, ne jouent pas exactement le même rôle.

En clair — Séparer les responsabilités

Dans un restaurant, la **salle** (prise de commande, service, caisse) et la **cuisine** (préparer le plat) sont deux métiers. On peut changer le cuisinier sans réorganiser la salle. De même, on peut remplacer `runc` par `crun` sans changer la façon dont les images sont gérées.

Répartition simplifiée

Le schéma classe les étapes en « gestion » et « exécution » pour faire comprendre l'idée. En réalité, plusieurs composants (Docker, containerd, runc, le réseau...) coopèrent, et le détail exact dépend de l'outil utilisé.

5. Architecture Docker

Docker repose sur plusieurs composants qui collaborent pour transformer une commande comme `docker run nginx` en un conteneur réellement exécuté par le kernel Linux. Chaque composant possède **une responsabilité différente**.

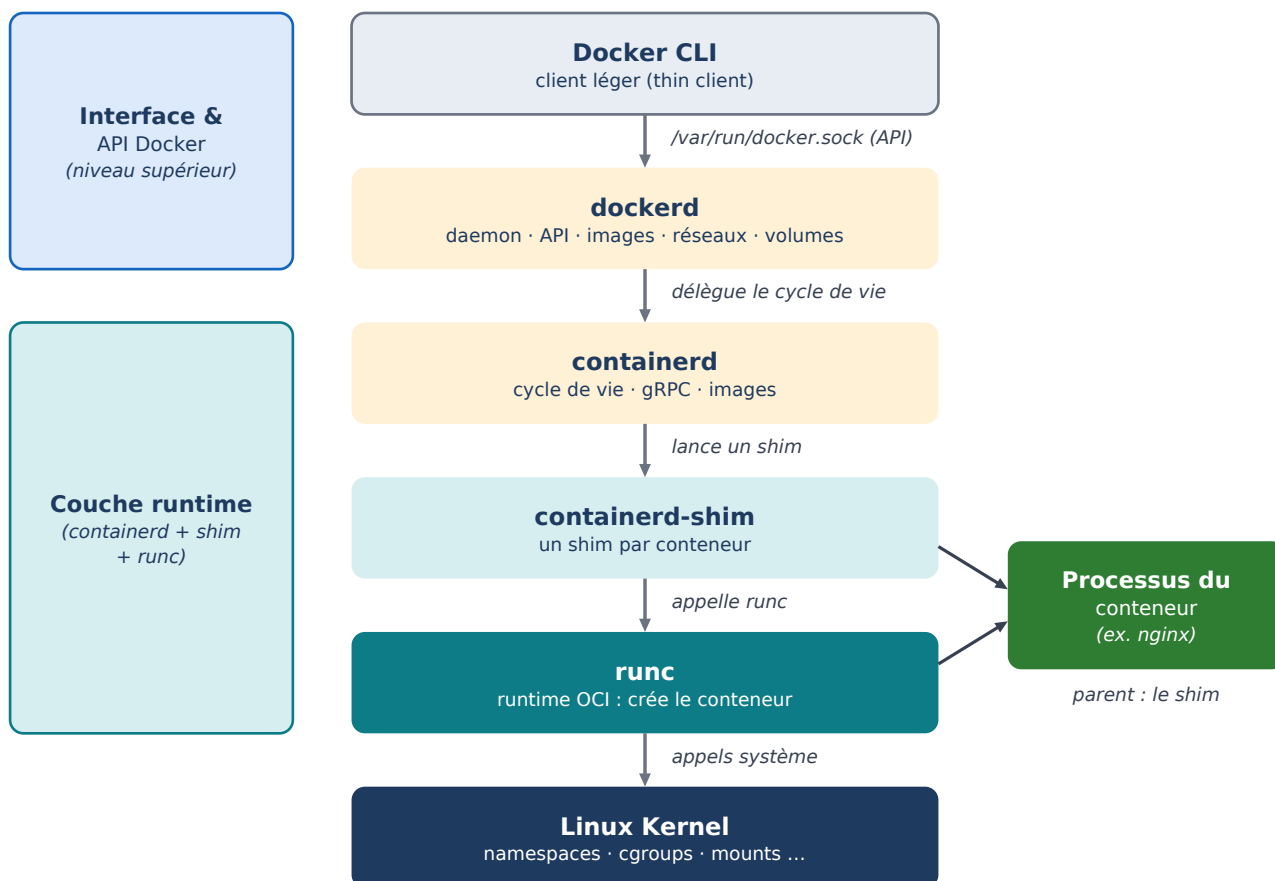


Figure 4 — L'architecture Docker, de la commande tapée jusqu'au kernel.

5.1. Docker CLI

Le **Docker CLI** est un **client léger** (*thin client*). Lorsque vous tapez :

```
docker run
docker ps
docker build
docker pull
docker stop
```

il transforme votre commande en **une requête API** et l'envoie via `/var/run/docker.sock` au Docker daemon.

Le CLI lui-même ne construit pas les images, ne télécharge pas les layers et ne démarre pas les conteneurs : il se contente de **transmettre les requêtes**. Le véritable travail est effectué par les composants situés derrière lui.

Cette séparation est importante car le CLI et le daemon n'ont même pas besoin d'être exécutés sur la même machine. Docker peut exposer son API à distance, permettant ainsi à des outils externes et à des systèmes d'automatisation de communiquer directement avec le daemon.

En clair — Socket Unix et API

Une **socket Unix** est un fichier spécial (`/var/run/docker.sock`) qui sert de **guichet** : le CLI y dépose sa demande, le daemon la lit et répond. C'est comme déposer un formulaire dans la boîte aux lettres du service concerné. Une **API** est simplement l'ensemble des demandes que ce guichet sait traiter.

5.2. dockerd

dockerd est le **daemon Docker**. Il reçoit les requêtes provenant du CLI et gère les **images**, les **réseaux**, les **volumes** et l'ensemble de l'API Docker.

Cependant, `dockerd` **n'exécute pas directement** les conteneurs. Il délègue la gestion de leur cycle de vie à `containerd`. Cette séparation permet aux conteneurs de continuer à fonctionner même si le daemon Docker redémarre : la couche runtime peut fonctionner indépendamment de la couche d'API Docker de niveau supérieur.

En clair — Daemon

Un **daemon** est un programme qui tourne **en permanence en arrière-plan**, comme un gardien d'immeuble toujours présent : on sonne (le CLI envoie une requête) et il s'occupe de la demande.

Nuance en pratique

L'architecture rend possible la survie des conteneurs, mais Docker n'active pas toujours ce comportement par défaut : pour que les conteneurs continuent de tourner quand `dockerd` redémarre, il faut en général activer l'option `live-restore` du daemon.

5.3. containerd

`containerd` est un composant spécialisé dans la **gestion du cycle de vie des conteneurs**. Il gère notamment :

- la gestion des **images** ;
- la création et la gestion des **conteneurs** ;
- la gestion des **tâches** (processus des conteneurs) ;
- la communication avec le **runtime OCI**, à travers une **API gRPC**.

Lorsque `dockerd` doit démarrer un conteneur, il délègue cette opération à `containerd`. À partir de ce moment, le daemon Docker se retire en grande partie du chemin d'exécution.

En 2017, Docker a donné `containerd` à la **Cloud Native Computing Foundation (CNCF)** en tant que projet indépendant. Aujourd'hui, **Kubernetes** communique avec `containerd` et d'autres runtimes compatibles avec **CRI**, plutôt qu'avec Docker. Docker est resté une plateforme destinée aux développeurs, tandis que `containerd` est devenu un runtime utilisé **sous** les plateformes d'orchestration.

Cependant, `containerd` **ne crée pas directement** les conteneurs : il transmet cette responsabilité au composant situé plus bas dans la pile.

En clair — gRPC, CRI et Kubernetes

gRPC est un protocole qui permet à un programme d'appeler des fonctions d'un autre programme comme s'il s'agissait de fonctions locales. **CRI** (*Container Runtime Interface*) est l'interface standard par laquelle **Kubernetes** (l'orchestrateur qui gère des milliers de conteneurs) parle aux runtimes. Tant qu'un runtime « parle CRI », Kubernetes peut l'utiliser.

5.4. containerd-shim

Le `containerd-shim` est un **processus léger** situé entre `containerd` et le conteneur en cours d'exécution. Avant qu'un conteneur ne démarre, `containerd` lance d'abord un processus shim. Le shim devient le **parent** du processus du conteneur et permet au conteneur de **continuer à fonctionner même si containerd rencontre un problème ou redémarre**. Chaque conteneur possède son propre shim.

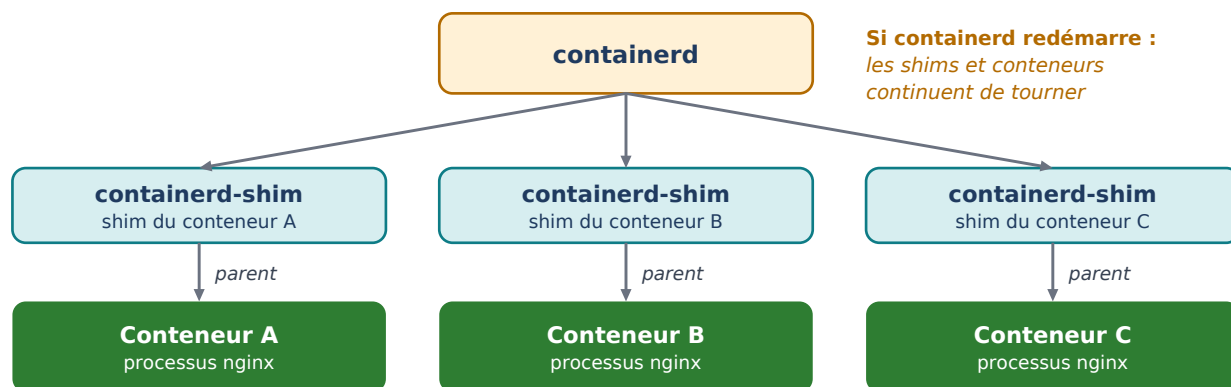


Figure 5 — Un shim par conteneur : il reste le parent du processus même si containerd redémarre.

En clair — Shim

Le shim est la **nounou** du conteneur : elle reste auprès de l'enfant même si les parents (containerd) s'absentent ou changent d'emploi du temps. *Shim* signifie littéralement « cale » : une petite pièce intercalée entre deux éléments.

5.5. runc

runc est le **runtime OCI** (*Open Container Initiative*) qui interagit **directement avec le kernel Linux** pour créer le conteneur. Il lit la configuration du conteneur et crée notamment :

- les **namespaces** ;
- les **cgroups**.

Il configure ensuite le système de fichiers et **démarré le processus** du conteneur. C'est à ce niveau que le conteneur cesse d'être simplement un objet géré par Docker et devient concrètement **un processus Linux**. Le kernel applique alors les mécanismes d'isolation du processus.

5.6. Linux Kernel

Le kernel Linux fournit les mécanismes fondamentaux utilisés pour isoler et contrôler les processus : **Namespaces, cgroups, mounts, capabilities, seccomp**... Le kernel est donc la couche qui fournit les **primitives** nécessaires à l'exécution des conteneurs.

5.7. Qui fait quoi ?

Composant	Rôle	Niveau
Docker CLI	Interface utilisateur (client léger)	Interface
<code>dockerd</code>	Daemon Docker : API, images, réseaux, volumes	API Docker
<code>containerd</code>	Gestion du cycle de vie des conteneurs	High-level
<code>containerd-shim</code>	Intermédiaire entre containerd et le processus	Runtime
<code>runc</code>	Exécution bas niveau selon OCI	Low-level
Linux Kernel	Isolation et contrôle des ressources	Système

À retenir

- ✓ Le CLI ne fait que **transmettre** des requêtes à `dockerd` via `/var/run/docker.sock`.
- ✓ `dockerd` gère l'API Docker mais **délègue** l'exécution à `containerd`.
- ✓ `containerd` gère le cycle de vie ; il lance un **shim par conteneur** puis `runc`.
- ✓ `runc` crée namespaces et cgroups, configure le filesystem et démarre le processus : c'est là que le conteneur devient **un processus Linux**.

5.8. Pour observer sur votre machine

Ajout pédagogique : ces commandes ne figurent pas dans le cours d'origine mais permettent de voir la chaîne en action.

```
# Versions des composants : Engine, containerd, runc ...
docker version

# Quel runtime OCI est utilisé ?
docker info | grep -i runtime

# Lancer un conteneur puis voir son shim parmi les processus de l'hôte
docker run -d --name demo nginx
ps -ef | grep containerd-shim

# Le CLI n'est qu'un client de l'API : même résultat avec curl via la socket
curl --unix-socket /var/run/docker.sock http://localhost/version

# Le CLI peut aussi parler à un daemon distant
DOCKER_HOST=ssh://utilisateur@serveur docker ps
```

6. Architecture Podman

Podman utilise également des **OCI runtimes** comme `runc` ou `crun`, mais son architecture présente une différence importante par rapport à Docker : **Podman est daemonless**. Il n'utilise pas de daemon central équivalent à `dockerd`.

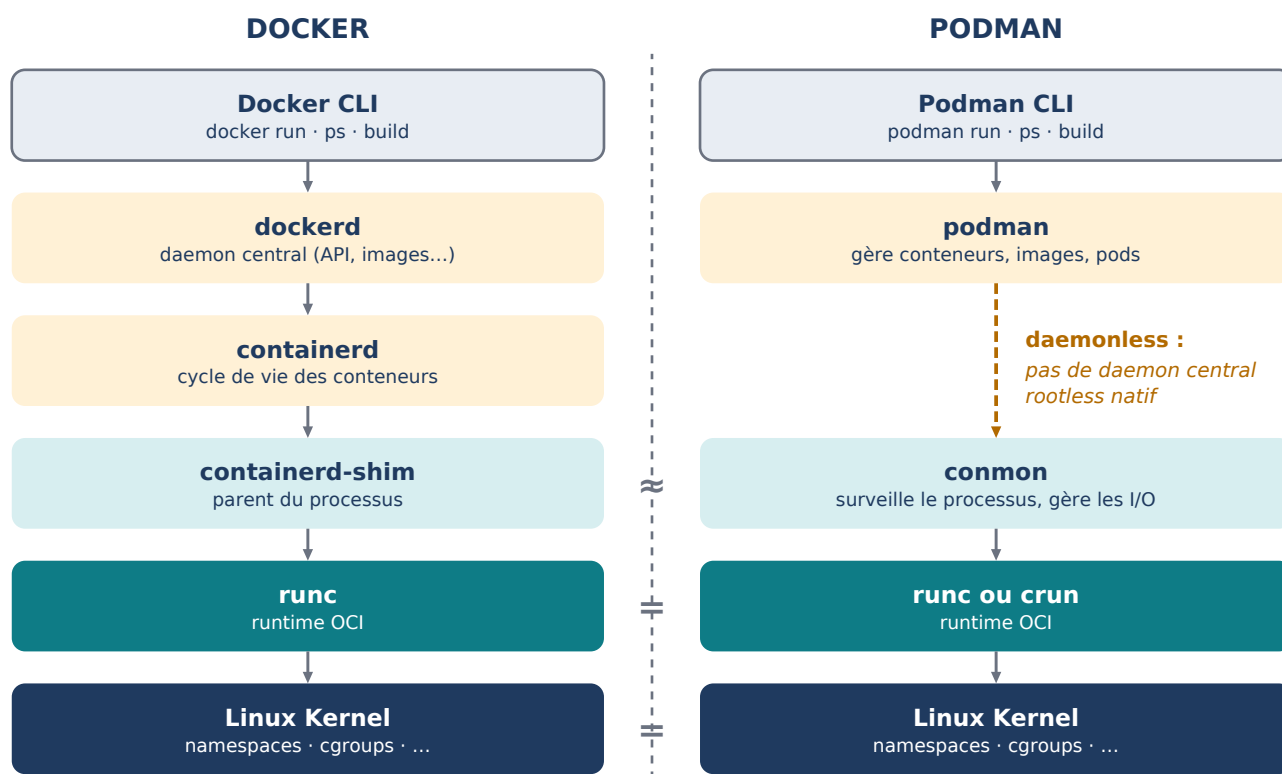


Figure 6 — Docker (avec daemon) et Podman (daemonless) : même base, architectures différentes.

En clair — Daemonless

Avec Docker, un **gardien permanent** (le daemon) reçoit toutes les demandes. Avec Podman, **pas de gardien** : chaque commande `podman` fait directement le travail puis laisse le conteneur vivre sa vie. Moins de pièce maîtresse privilégiée à protéger, et pas de processus central dont tout dépend.

6.1. Podman CLI

Le Podman CLI est l'interface utilisée par l'utilisateur : `podman run`, `podman ps`, `podman build`, `podman pull`, `podman stop`. Contrairement à Docker, la commande `podman` **ne nécessite pas de communiquer avec un daemon central permanent**.

6.2. Podman

Podman assure directement la gestion des opérations liées aux conteneurs. Il peut notamment :

- créer des conteneurs ;
- gérer leur cycle de vie ;
- gérer les images ;
- gérer les **pods** ;
- préparer l'environnement nécessaire à l'exécution ;
- invoquer un runtime OCI.

```
podman
|
▼
OCI Runtime
|
▼
Linux Kernel
```

En clair — Pod

Un **pod** est un groupe de conteneurs qui partagent certaines ressources (notamment le réseau) et sont gérés ensemble, comme des colocataires d'un même appartement. Le concept vient de Kubernetes.

6.3. common

`common` (*container monitor*) est utilisé par Podman pour **surveiller les processus des conteneurs**. Il peut notamment :

- surveiller le processus du conteneur ;
- gérer les entrées/sorties (I/O) ;
- conserver les informations nécessaires au suivi du conteneur ;
- permettre au processus du conteneur de **continuer indépendamment** de la commande Podman qui l'a lancé.

```
Podman
|
▼
common
|
▼
runc / crun
|
▼
Linux Kernel
```

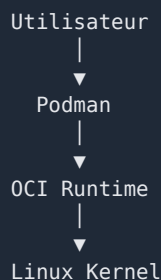
On peut voir `common` comme l'équivalent du `containerd-shim` côté Docker : un petit processus qui reste auprès du conteneur.

6.4. runc ou crun

Podman peut utiliser différents OCI runtimes, par exemple `runc` ou `crun`. Ces runtimes ont pour rôle d'**exécuter réellement** le processus du conteneur en utilisant les mécanismes du kernel Linux.

6.5. Rootless

Une caractéristique importante de Podman est sa capacité à fonctionner en **rootless** : un utilisateur peut exécuter des conteneurs **sans nécessairement disposer des privilèges root**.



Cela permet notamment de **réduire la dépendance à un daemon privilégié central**.

En clair — Rootless

Grâce au **user namespace** vu au chapitre précédent, le processus est « root » **dans** le conteneur mais n'est qu'un utilisateur ordinaire **sur** l'hôte. Même en cas de faille, l'attaquant n'hérite que des droits limités de cet utilisateur.

À retenir

- ✓ Docker repose sur un **daemon central** (`dockerd`) ; Podman est **daemonless**.
- ✓ Les deux peuvent utiliser un **OCI runtime** (`runc`, et `crun` pour Podman) pour exécuter les conteneurs.
- ✓ `common` joue chez Podman un rôle comparable à `containerd-shim` chez Docker.

7. Différence entre Docker et Podman

Docker et Podman fournissent tous les deux des outils permettant de **créer, exécuter et gérer** des conteneurs. La principale différence étudiée dans ce chapitre concerne leur **architecture** et leur **mode de fonctionnement**.

Élément	Docker	Podman
Interface CLI	<code>docker</code>	<code>podman</code>
Daemon central	<code>dockerd</code>	Pas de daemon central
Gestion des conteneurs	<code>dockerd</code> + <code>containerd</code>	Podman
Runtime OCI	<code>runc</code>	<code>runc</code> ou <code>crun</code>
Shim	<code>containerd-shim</code>	<code>common</code>
Mode rootless	Possible	Pris en charge nativement

En résumé

Docker utilise une architecture basée sur un **daemon central**, avec `dockerd` et `containerd` qui coordonnent la gestion des conteneurs.

Podman utilise une architecture **daemonless** : les commandes sont exécutées directement par Podman et les conteneurs peuvent être exécutés avec un runtime OCI comme `runc` ou `crun`.

Dans les deux cas, l'exécution bas niveau repose finalement sur les mécanismes fournis par le **Linux Kernel**.

8. OCI — Open Container Initiative

Maintenant qu'on comprend la notion de runtime, une autre question apparaît :

La question

Comment différents runtimes peuvent-ils fonctionner selon des règles communes ?

C'est là qu'intervient l'**OCI**.

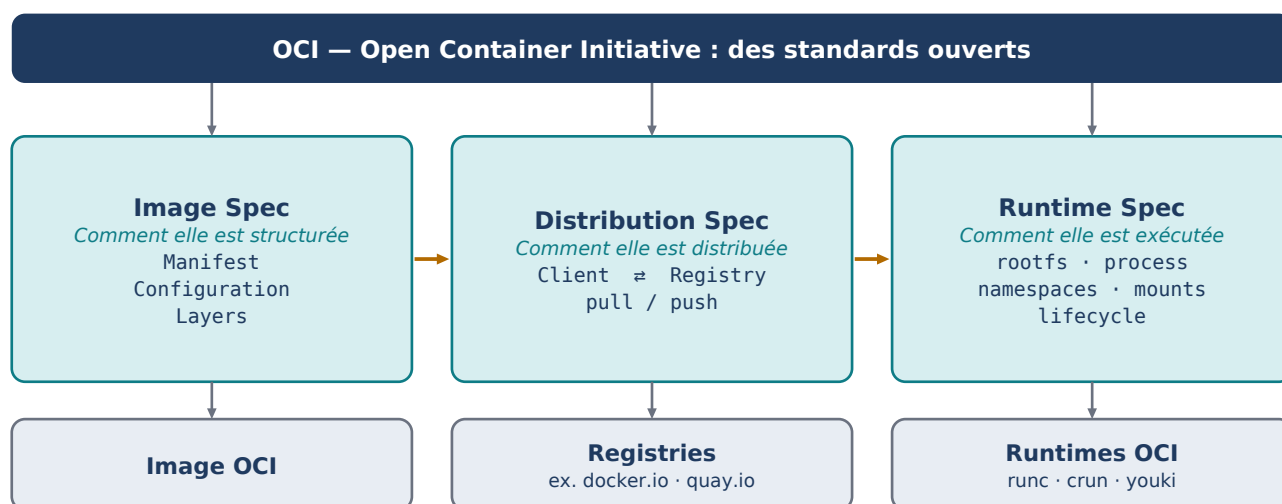
8.1. Définition

OCI (Open Container Initiative) définit des **standards ouverts** pour les images, les runtimes et la distribution des artefacts de conteneurs. C'est un ensemble de **spécifications communes**.

En clair — OCI

Les **conteneurs maritimes** ont des dimensions normalisées : n'importe quel bateau, camion ou grue du monde peut les manipuler sans se soucier de leur contenu. L'OCI joue le même rôle pour les conteneurs logiciels : une image conforme peut être lue et exécutée par **n'importe quel outil conforme** (Docker, Podman, containerd, CRI-O...).

8.2. Les trois spécifications OCI



De la fabrication de l'image à son exécution : trois contrats complémentaires

Figure 7 — Les trois spécifications OCI : structurer, distribuer, exécuter.

Documentation officielle : <https://specs.opencontainers.org/>

8.2.1. OCI Image Specification

Elle définit le **format standard d'une image** de conteneur. Conceptuellement :

```

Image
├── Manifest
├── Configuration
└── Layers
  
```

Élément	En clair
Manifest	La table des matières de l'image : il liste la configuration et les layers qui la composent.
Configuration	La fiche technique : architecture, variables d'environnement, commande par défaut...
Layers	Les couches de fichiers empilées qui forment le contenu de l'image.

8.2.2. OCI Runtime Specification

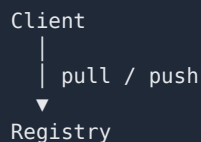
Elle définit le **modèle standard d'exécution** d'un conteneur. Elle concerne notamment : le root filesystem, le processus, les paramètres d'exécution, les namespaces, les mounts, les paramètres Linux et le lifecycle.

Elle est notamment implémentée par des runtimes comme [runc](#), [crun](#) et [youki](#).

8.2.3. OCI Distribution Specification

Elle définit les règles permettant aux **clients** et aux **registries** d'échanger des artefacts OCI.

Conceptuellement :



En clair — Registry

Un **registry** est une **bibliothèque d'images** en ligne (Docker Hub, quay.io...). **pull** = emprunter une image, **push** = en déposer une.

8.2.4. Ne pas confondre

Spécification	Question à laquelle elle répond
Image Specification	Comment l'image est structurée ?
Distribution Specification	Comment elle est distribuée ?
Runtime Specification	Comment elle est exécutée ?

À retenir

- ✓ L'OCI est un ensemble de **standards ouverts** qui permet aux outils de conteneurs d'être interchangeables.
- ✓ Trois spécifications : **Image, Runtime, Distribution**.

Ce qu'il faut absolument retenir

Container Runtime

Un logiciel chargé de gérer et/ou d'exécuter des conteneurs.

High-Level Runtime

Gère les images, les conteneurs, les snapshots, les tâches et le cycle de vie.

Low-Level Runtime

Crée et exécute réellement le processus du conteneur en utilisant les mécanismes du système d'exploitation.

OCI

Définit les standards ouverts utilisés par l'écosystème des conteneurs.

containerd

Un high-level container runtime qui gère le cycle de vie des conteneurs et s'appuie sur un runtime OCI pour leur exécution.

CRI-O

Un high-level container runtime conçu pour l'intégration Kubernetes via CRI.

runc

Un low-level OCI runtime qui exécute les processus des conteneurs.

Linux Kernel

Fournit les mécanismes fondamentaux d'isolation et de contrôle.

La synthèse en 4 phrases

1. Un **runtime high-level** (containerd, CRI-O) **gère** : images, stockage, cycle de vie.
2. Un **runtime low-level** (runc, crun, youki) **exécute** : il crée le processus isolé avec le kernel.
3. **Docker** = CLI → dockerd → containerd → shim → runc ; **Podman** = podman → conmon → runc/crun (sans daemon).
4. L'**OCI** standardise les images, leur distribution et leur exécution : tout est interchangeable.

Glossaire des termes importants

Terme	Définition simple
Container runtime	Logiciel qui gère et/ou exécute des conteneurs en s'appuyant sur le kernel.
High-level runtime	Runtime qui gère images, stockage et cycle de vie (containerd, CRI-O).
Low-level runtime	Runtime qui crée et exécute le processus isolé (runc, crun, youki).
Daemon	Programme qui tourne en permanence en arrière-plan.
Daemonless	Architecture sans daemon central : chaque commande fait directement le travail (Podman).
<code>docker.sock</code>	Socket Unix par laquelle le Docker CLI envoie ses requêtes API au daemon.
API	Ensemble des demandes qu'un programme sait traiter, utilisable par d'autres programmes.
gRPC	Protocole d'appel de fonctions à distance, utilisé par containerd.
CRI	Container Runtime Interface : interface par laquelle Kubernetes parle aux runtimes.
CNCF	Cloud Native Computing Foundation : fondation qui héberge containerd et Kubernetes.
Kubernetes	Orchestrateur qui déploie et supervise de nombreux conteneurs.
Shim	Petit processus intermédiaire : parent du processus du conteneur (containerd-shim).
<code>common</code>	Moniteur de conteneur de Podman : surveille le processus et gère ses entrées/sorties.
<code>runc, crun, youki</code>	Runtimes OCI bas niveau (runc en Go, crun en C, youki en Rust).
OCI	Open Container Initiative : standards ouverts pour images, runtimes et distribution.
Manifest	Document qui décrit le contenu d'une image (configuration + layers).
Layer	Couche de fichiers d'une image.
Registry	Serveur qui stocke et distribue des images (pull / push).
Snapshot	Photo instantanée d'un système de fichiers, utilisée pour assembler le filesystem d'un conteneur.
Capabilities	Droits du root découpés en privilèges précis.
seccomp	Filtre des appels système autorisés pour un processus.
Rootless	Exécution de conteneurs sans privilèges root sur l'hôte.
Pod	Groupe de conteneurs gérés ensemble et partageant certaines ressources.

Questions de compréhension — avec corrigés

Essayez de répondre de vous-même avant de lire la correction.

Question 1 Qu'est-ce qu'un Container Runtime ?

Correction

Un **container runtime** est un logiciel responsable de la **gestion et/ou de l'exécution** des conteneurs, en utilisant les mécanismes fournis par le système d'exploitation (namespaces, cgroups, mounts...).

C'est lui qui met en place ces mécanismes quand on demande de lancer un conteneur.

Question 2 Quelle est la différence entre un High-Level et un Low-Level Container Runtime ?

Correction

Le **high-level** gère les aspects généraux : **images, stockage, snapshots, cycle de vie** (`containerd`, `CRI-O`). Il **gère et orchestre**.

Le **low-level** est proche du système et **crée et exécute** le processus du conteneur en utilisant namespaces, cgroups, mounts, capabilities et seccomp (`runc`, `crun`, `youki`). Il **exécute et isole**.

Question 3 Citer deux exemples de High-Level Container Runtime.

Correction

`containerd` et `CRI-O`.

Question 4 Citer deux exemples de Low-Level Container Runtime.

Correction

`runc` et `crun` (on peut aussi citer `youki`).

Question 5 Quel est le rôle de l'OCI (Open Container Initiative) ?

Correction

L'OCI définit des **standards ouverts** pour les images, les runtimes et la distribution des artefacts de conteneurs. Grâce à ces **spécifications communes**, différents outils et runtimes fonctionnent selon les mêmes règles et sont interchangeables.

Question 6 Quelles sont les trois principales spécifications OCI ?

Correction

Image Specification : comment l'image est structurée (manifest, configuration, layers).

Runtime Specification : comment le conteneur est exécuté (rootfs, processus, namespaces, mounts, lifecycle...).

Distribution Specification : comment clients et registries échangent les artefacts (pull / push).

Question 7 Quelle est la différence entre containerd et runc ?

Correction

`containerd` est un **high-level runtime** : il gère les images, les conteneurs, les tâches et leur cycle de vie, et expose une API gRPC. Il **ne crée pas lui-même** le processus isolé.

`runc` est un **low-level OCI runtime** : il lit la configuration, crée les **namespaces** et **cgroups**, configure le filesystem et **démarré le processus** du conteneur en dialoguant avec le kernel.

`containerd` délègue donc à `runc` (via un `containerd-shim`) la création effective du conteneur.

Question 8 Quelle est la différence entre docker et podman ?**Correction**

Docker repose sur un **daemon central** : le CLI envoie des requêtes à `dockerd`, qui s'appuie sur `containerd`, `containerd-shim` puis `runc`.

Podman est **daemonless** : la commande `podman` gère directement les conteneurs, utilise `conmon` comme moniteur et un runtime OCI (`runc` ou `crun`). Il prend en charge nativement le mode **rootless**.

Dans les deux cas, l'exécution bas niveau repose sur le **Linux Kernel**.

Préparation pour la suite

L'objectif du lab suivant sera d'**explorer progressivement le fonctionnement de containerd**, de la gestion des images jusqu'au lancement et à l'inspection d'un conteneur, afin de comprendre les composants qui interviennent réellement sous le capot.

Avant le lab, vérifiez que vous savez répondre à :

- Quel composant télécharge et stocke les images ? (high-level)
- Quel composant crée réellement le processus isolé ? (low-level)
- Qui est le parent du processus d'un conteneur Docker ? (le shim)